



Task 1: Build a Web API

- **Objective:** Create a RESTful API that performs CRUD operations on simple entities like "Orders", "Customers"
- **Requirements:**
 1. The API should have the following endpoints:
 - GET /api/customers: Retrieve all customers.
 - GET /api/customers/{id}: Retrieve a customer by ID.
 - POST /api/customers: Add a new customer.
 - PUT /api/customers/{id}: Update an existing customer.
 - DELETE /api/customers/{id}: Delete a customer.
 2. Customer entity should have the minimum data about the customer like (first_name, last_name, address, email, phone) etc.
 3. Create also all CRUD operations on Orders entity.
 4. Assume Customers and Orders have M-N relationship total participation.
 5. Use SQL server database for data storage.
 6. Use .NET 4.8 framework for the Web API and Entity Framework 6.x as an ORM.
 7. Validate inputs using data annotations and Fluent validations (use the two approaches in your task) and feel free to assume any validation logic.
 8. Log the request and response in case of failure and success requests (File or DB logs).
 9. Use proper exception handling techniques to handle exceptions.
 10. Implement Dependency Injection to make your actions depends on abstraction not concrete classes.
 11. Implement login logic using JWT authentication claims to validate the user and validate the token and claims on each request. Protect specific endpoints with role-based authorization. Assume you have two roles ('user', and 'admin').
- **Deliverables**
 - Source code for the all endpoints and services used with them.
 - Must use .NET 4.8 framework for developing this task.
 - Postman collection and examples for each endpoint.
 - Brief documentation explaining the purpose of each object.

Task 2: SQL-Server Database

- **Objective:** Build a database system for managing employee performance, including features like tracking employee details, performance reviews, department statistics, and dynamic reporting.
- **Requirements:**

1. Create Tables

Employees:

- EmployeeID (Primary Key, INT, Identity)
- FirstName (NVARCHAR(50))
- LastName (NVARCHAR(50))
- DateOfJoining (DATE)
- DepartmentID (Foreign Key)
- IsActive (BIT)

Departments:

- DepartmentID (Primary Key, INT, Identity)
- DepartmentName (NVARCHAR(50))

PerformanceReviews:

- ReviewID (Primary Key, INT, Identity)
- EmployeeID (Foreign Key)
- ReviewDate (DATE)
- Score (INT)
- Comments (NVARCHAR(255))

2. Views

- Create a view `vw_EmployeeDetails` to display employee details with their department names.

3. Indexes

- Add a non-clustered index on `DateOfJoining` in the `Employees` table for optimized queries.
- Add a composite index on `EmployeeID` and `ReviewDate` in the `PerformanceReviews` table.

4. Functions

- Create a scalar function `fn_GetExperience` that calculates the experience of an employee in years based on `DateOfJoining`.
- Create a table-valued function `fn_GetTopPerformers` to return the top N employees based on average performance score

5. Stored Procedures

- Procedure 1: `usp_AddEmployee` Add a new employee with details.
- Procedure 2: `usp_AddPerformanceReview` Add a new performance review for an employee.
- Procedure 3: `usp_GetDepartmentStatistics` Return the average performance score and the count of employees in each department.
- Procedure 4: `usp_GetDynamicReport` Accept parameters for `DepartmentID`, `MinimumScore`, and `StartDate/EndDate` to filter employees and their performance reviews dynamically.

6. Triggers

- Create a trigger `trg_UpdateEmployeeStatus` on the `PerformanceReviews` table to mark an employee as inactive (`IsActive = 0`) if their average score drops below 3.

• Deliverables

- Scripts for creating tables, views, functions, procedures, and triggers.
- Scripts for inserting sample data.
- Results from running the dynamic reporting procedure with different parameters.
- Brief documentation explaining the purpose of each database object.